

SmartCampusManager

Database Relations, Schema & Problem Solving Purpose

Oracle XE · campus_admin schema · localhost:1522/xe

Course: SEng2022 — Design and Analysis of Data Structures and Algorithms
Debre Berhan University

1. Schema Overview

The database has 7 tables, 4 sequences, and 2 stored procedures. All primary keys are auto-generated by Oracle sequences.

Table	Purpose	PK Sequence
USERS	All system users: Admin, Technician, Security, Auditor, Student	SEQ_USERS
ASSETS	University physical assets (equipment, furniture, etc.)	SEQ_ASSETS
MAINTENANCE_LOG	Repair/maintenance requests linked to assets	—
STUDENT_ITEMS	Personal items registered by students	SEQ_STUDENT_ITEMS
LOST_FOUND	Lost or found item reports	SEQ_LOST_FOUND
LOCATIONS	Campus buildings and rooms	—
AUDIT_LOG	System-level event trail	—

2. Entity-Relationship Summary

Parent Table	Relation	Child Table	Foreign Key Column
USERS	1 — M	ASSETS	ASSETS.CREATED_BY
USERS	1 — M	MAINTENANCE_LOG	MAINTENANCE_LOG.REPORTED_BY
USERS	1 — M	STUDENT_ITEMS	STUDENT_ITEMS.STUDENT_ID
USERS	1 — M	LOST_FOUND	LOST_FOUND.REPORTED_BY
ASSETS	1 — M	MAINTENANCE_LOG	MAINTENANCE_LOG.ASSET_ID
LOCATIONS	1 — M	STUDENT_ITEMS	STUDENT_ITEMS.LOCATION_ID
LOCATIONS	1 — M	LOST_FOUND	LOST_FOUND.FOUND_LOCATION

3. Table Definitions

TABLE: USERS (PK sequence: SEQ_USERS)		
Column	Type	Constraint / Notes
USER_ID	NUMBER	PRIMARY KEY
USERNAME	VARCHAR2(50)	UNIQUE, NOT NULL
FULL_NAME	VARCHAR2(100)	NOT NULL
PASSWORD_HASH	VARCHAR2(64)	SHA-256 hex
ROLE	VARCHAR2(20)	ADMIN TECHNICIAN SECURITY AUDITOR STUDENT
EMAIL	VARCHAR2(100)	nullable
PHONE	VARCHAR2(20)	nullable
DEPARTMENT	VARCHAR2(100)	nullable
STUDENT_ID	VARCHAR2(30)	university student ID (STUDENT role only)
IS_ACTIVE	CHAR(1)	'Y' active / 'N' deactivated
CREATED_AT	DATE	DEFAULT SYSDATE

Note: Central auth table. All FK user references point here.

TABLE: ASSETS (PK sequence: SEQ_ASSETS)		
Column	Type	Constraint / Notes
ASSET_ID	NUMBER	PRIMARY KEY
ASSET_NAME	VARCHAR2(100)	NOT NULL
CATEGORY	VARCHAR2(50)	e.g. Furniture, Electronics, Vehicle
SERIAL_NUMBER	VARCHAR2(100)	nullable — indexed in DSA BST
PURCHASE_VALUE	NUMBER(12,2)	monetary value of the asset
CONDITION	VARCHAR2(30)	GOOD BROKEN UNDER_MAINTENANCE DECOMMISSIONED
CREATED_BY	NUMBER	FK → USERS.USER_ID

Note: Sorted by Bubble Sort (ID/value) and Merge Sort (name/value) in AssetWindow.

TABLE: MAINTENANCE_LOG (PK sequence: — own PK)		
Column	Type	Constraint / Notes
MAINTENANCE_ID	NUMBER	PRIMARY KEY
ASSET_ID	NUMBER	FK → ASSETS.ASSET_ID (NOT NULL)
REPORTED_BY	NUMBER	FK → USERS.USER_ID
DESCRIPTION	VARCHAR2(500)	issue description
STATUS	VARCHAR2(20)	PENDING IN_PROGRESS RESOLVED
PRIORITY	VARCHAR2(10)	URGENT HIGH NORMAL LOW

Note: Greedy algorithm reads PRIORITY column to auto-score and sort rows.

TABLE: LOCATIONS (PK sequence: — own PK)		
Column	Type	Constraint / Notes
LOCATION_ID	NUMBER	PRIMARY KEY
BUILDING_NAME	VARCHAR2(100)	e.g. Library, Admin Block
ROOM_NUMBER	VARCHAR2(20)	e.g. 101, Lab-3

Note: Shared lookup table used by both STUDENT_ITEMS and LOST_FOUND.

TABLE: STUDENT_ITEMS (PK sequence: SEQ_STUDENT_ITEMS)		
Column	Type	Constraint / Notes
ITEM_ID	NUMBER	PRIMARY KEY
STUDENT_ID	NUMBER	FK → USERS.USER_ID
ITEM_NAME	VARCHAR2(100)	e.g. Laptop, Bag
CATEGORY	VARCHAR2(50)	e.g. Electronics, Clothing
BRAND	VARCHAR2(50)	nullable
MODEL	VARCHAR2(50)	nullable
SERIAL_NUMBER	VARCHAR2(100)	nullable — all loaded into BST at login
STATUS	VARCHAR2(20)	REGISTERED OUT_OF_CAMPUS LOST STOLEN
LOCATION_ID	NUMBER	FK → LOCATIONS.LOCATION_ID (nullable)

Note: SecurityCheckWindow loads all serial numbers into a BST for $O(\log n)$ lookup.

TABLE: LOST_FOUND (PK sequence: SEQ_LOST_FOUND)		
Column	Type	Constraint / Notes
RECORD_ID	NUMBER	PRIMARY KEY
REPORTED_BY	NUMBER	FK → USERS.USER_ID
RECORD_TYPE	VARCHAR2(10)	LOST FOUND
ITEM_DESCRIPTION	VARCHAR2(500)	text description
FOUND_LOCATION	NUMBER	FK → LOCATIONS.LOCATION_ID
REPORTED_AT	DATE	DEFAULT SYSDATE

TABLE: AUDIT_LOG (PK sequence: — own PK)		
Column	Type	Constraint / Notes
LOG_ID	NUMBER	PRIMARY KEY
USER_ID	NUMBER	FK → USERS.USER_ID
ACTION	VARCHAR2(100)	e.g. LOGIN, DELETE_ASSET
TABLE_NAME	VARCHAR2(50)	affected table
RECORD_ID	NUMBER	affected row PK
LOG_TIME	DATE	DEFAULT SYSDATE

Note: Auto-populated by triggers or application code.

4. Stored Procedures

Procedure	Parameters	Action
SP_REPORT_ISSUE	p_asset_id, p_reported_by, p_description, p_priority	Inserts a new row into MAINTENANCE_LOG with STATUS='PENDING'
SP_RESOLVE_MAINTENANCE	p_maintenance_id, p_resolved_by, p_note	Sets STATUS='RESOLVED', records resolver and note

5. Sequences

Sequence	Table	Create command
SEQ_USERS	USERS	CREATE SEQUENCE SEQ_USERS START WITH 1 INCREMENT BY 1;
SEQ_ASSETS	ASSETS	CREATE SEQUENCE SEQ_ASSETS START WITH 1 INCREMENT BY 1;
SEQ_STUDENT_ITEMS	STUDENT_ITEMS	CREATE SEQUENCE SEQ_STUDENT_ITEMS START WITH 1 INCREMENT BY 1;
SEQ_LOST_FOUND	LOST_FOUND	CREATE SEQUENCE SEQ_LOST_FOUND START WITH 1 INCREMENT BY 1;

6. Useful Oracle Commands for Testing

Connect

```
sqlplus campus_admin/your_password@localhost:1522/xe
```

Check all tables

```
SELECT table_name FROM user_tables ORDER BY table_name;
```

Count rows in each table

```
SELECT COUNT(*) FROM USERS;
SELECT COUNT(*) FROM ASSETS;
SELECT COUNT(*) FROM MAINTENANCE_LOG;
SELECT COUNT(*) FROM STUDENT_ITEMS;
SELECT COUNT(*) FROM LOST_FOUND;
SELECT COUNT(*) FROM LOCATIONS;
SELECT COUNT(*) FROM AUDIT_LOG;
```

Reset sequences

```
DROP SEQUENCE SEQ_USERS; CREATE SEQUENCE SEQ_USERS START WITH 1 INCREMENT BY 1;
```

```
DROP SEQUENCE SEQ_ASSETS; CREATE SEQUENCE SEQ_ASSETS START WITH 1 INCREMENT BY 1;
DROP SEQUENCE SEQ_STUDENT_ITEMS; CREATE SEQUENCE SEQ_STUDENT_ITEMS START WITH 1 INCREMENT BY 1;
DROP SEQUENCE SEQ_LOST_FOUND; CREATE SEQUENCE SEQ_LOST_FOUND START WITH 1 INCREMENT BY 1;
```

Truncate all data (child tables first)

```
TRUNCATE TABLE AUDIT_LOG;
TRUNCATE TABLE LOST_FOUND;
TRUNCATE TABLE STUDENT_ITEMS;
TRUNCATE TABLE MAINTENANCE_LOG;
TRUNCATE TABLE ASSETS;
TRUNCATE TABLE USERS;
TRUNCATE TABLE LOCATIONS;
```

Always truncate child tables before parent tables to respect FK constraints.

Test stored procedures

```
BEGIN SP_REPORT_ISSUE(1, 1, 'Projector broken', 'HIGH'); END;
BEGIN SP_RESOLVE_MAINTENANCE(1, 1, 'Replaced lamp'); END;
```

Maintenance log with joins

```
SELECT m.MAINTENANCE_ID, a.ASSET_NAME, u.FULL_NAME, m.STATUS, m.PRIORITY
FROM MAINTENANCE_LOG m
JOIN ASSETS a ON m.ASSET_ID = a.ASSET_ID
JOIN USERS u ON m.REPORTED_BY = u.USER_ID
ORDER BY m.MAINTENANCE_ID;
```

Student items with owner name

```
SELECT si.ITEM_ID, u.FULL_NAME, si.ITEM_NAME, si.STATUS, si.SERIAL_NUMBER
FROM STUDENT_ITEMS si
JOIN USERS u ON si.STUDENT_ID = u.USER_ID
ORDER BY si.ITEM_ID;
```

7. How Each DSA Connects to the Database

Algorithm	Ch.	Table	Connection
Bubble Sort	2	ASSETS	Sorts fetched rows by ASSET_ID or PURCHASE_VALUE in memory
Selection Sort	2	ASSETS	Sorts rows by ASSET_NAME or LOCATION in memory
Linked List	3	All tables	Records every DB INSERT/UPDATE/DELETE as a linked list node
Stack (Undo/Redo)	4	ASSETS	Saves asset snapshot before every DB write for undo
Recursion	5	STUDENT_ITEMS	BST insert/search/height helpers use recursive calls
BST	6	STUDENT_ITEMS	Loads all SERIAL_NUMBERS from DB into tree for $O(\log n)$ search
Graph BFS/DFS	7	— in-memory	Campus buildings modelled as graph nodes after real DBU map
Merge Sort	8	ASSETS	Faster sort $O(n \log n)$ vs Bubble Sort $O(n^2)$ on same data
Greedy	9	MAINTENANCE_LOG	Reads PRIORITY column, scores each row, sorts desc by score

8. Problem Solving Purpose of Each DSA

This section explains the real problem each data structure or algorithm solves. For each one: what was broken or slow before, what DSA was applied, and what improvement it delivers.

Ch. 1 — Big-O Complexity Analysis	
Problem	Without complexity analysis, there is no way to justify why one algorithm was chosen over another. The instructor cannot verify the
Solution	Every function in dsaengine.cpp has a header comment showing Best / Average / Worst time complexity and Space complexity. E
Result	Academic justification for every algorithm choice. The instructor can read the Big-O directly from the source code.
Code	dsaengine.cpp — comment block above every function

Ch. 2 — Bubble Sort & Selection Sort	
Problem	The ASSETS table can have hundreds of rows. The user needs to sort by ID, value, name, or location. Firing a new ORDER BY S
Solution	After the initial DB fetch, rows are stored in a <code>std::vector<AssetRecord></code> . Bubble Sort compares adjacent pairs and bubbles the lar
Result	Instant sort switching in the UI. No network cost after initial load. Both algorithms demonstrated side-by-side for comparison.
Code	dsaengine.cpp: <code>bubbleSortById()</code> , <code>bubbleSortByValue()</code> , <code>selectionSortByName()</code> , <code>selectionSortByLocation()</code>

Ch. 3 — Singly Linked List — Action Log	
Problem	The system needs a FIFO log to record every DB operation (add/edit/delete asset). <code>std::queue</code> uses a deque internally and pre-all
Solution	A custom singly linked list replaces <code>std::queue</code> . Each <code>LLNode</code> holds exactly one string and a next pointer. <code>llEnqueue()</code> appends at
Result	Exact-memory FIFO action log. Every DB write is recorded. <code>llContents()</code> can print the full log at any time.
Code	dsaengine.h: <code>struct LLNode</code> dsaengine.cpp: <code>llEnqueue()</code> , <code>llDequeue()</code> , <code>llContents()</code>

Ch. 4 — Stack — Undo / Redo	
Problem	When an admin edits or deletes an asset, a mistake cannot be recovered without undo. Once the DB is updated, the previous stat
Solution	Before every DB write, the current asset state is saved as an <code>AssetSnapshot</code> and pushed onto <code>m_undoStack</code> (LIFO). Undo pops th
Result	Full undo/redo for all asset operations. $O(1)$ push and pop — no performance cost regardless of history size.
Code	dsaengine.cpp: <code>pushUndo()</code> , <code>popUndo()</code> , <code>pushRedo()</code> , <code>popRedo()</code> assetwindow.cpp: undo/redo buttons

Ch. 5 — Recursion	
Problem	Tree operations (insert, search, height, in-order, clear) must visit sub-trees. An iterative approach requires a manual stack, comple
Solution	All BST helpers are recursive: <code>bstInsertHelper()</code> recurses left/right until finding a null slot. <code>bstSearchHelper()</code> recurses into the corr
Result	Clean, short, readable tree code. Each function maps directly to its textbook definition. Recursion is demonstrated in both BST an
Code	dsaengine.cpp: <code>bstInsertHelper()</code> , <code>bstSearchHelper()</code> , <code>bstHeightHelper()</code> , <code>bstClearHelper()</code> , <code>bstInorderHelper()</code> , <code>graphDFS()</code> lambda

Ch. 6 — Binary Search Tree — Serial Number Index	
Problem	<code>SecurityCheckWindow</code> must check if a student's serial number exists in <code>STUDENT_ITEMS</code> . A linear scan (check each row) is $O(n)$
Solution	At window load, all serial numbers are fetched once from <code>STUDENT_ITEMS</code> and inserted into a BST. Insert is $O(\log n)$ average. A
Result	500 items: linear scan = up to 500 comparisons. BST search = ~ 9 comparisons ($\log_2(500) = 8.97$). More than 50x fewer comparis
Code	dsaengine.cpp: <code>bstInsert()</code> , <code>bstSearch()</code> securitycheckwindow.cpp: <code>loadSerialsBST()</code> , <code>searchItem()</code>

Ch. 7 — Graph — BFS & DFS Campus Navigation	
Problem	The campus has buildings connected by paths. A plain list of buildings cannot answer: 'What is the shortest route from the Main G
Solution	Buildings are graph nodes; paths are undirected edges in an adjacency list. BFS uses a queue to explore level by level — guaran
Result	BFS gives the exact shortest path between any two buildings. DFS gives the full reachability list from any starting point. Both resu
Code	dsaengine.cpp: graphInit(), graphBFS(), graphDFS() campusgraphwindow.cpp: runBFS(), runDFS()

Ch. 8 — Merge Sort — Divide and Conquer	
Problem	Bubble Sort and Selection Sort are $O(n^2)$. With 1000 assets that is ~1,000,000 comparisons. The UI would visibly freeze. A faster
Solution	Merge Sort divides the asset vector in half recursively until sub-arrays of size 1 (base case), then merges sorted halves. Recurren
Result	1000 assets: Bubble Sort ~1,000,000 comparisons. Merge Sort ~10,000 comparisons. 100x fewer operations. The divide-and-con
Code	dsaengine.cpp: mergeSortByValue(), mergeSortByName(), mergeSortHelperByValue/Name(), mergeByValue/Name() assetwind

Ch. 9 — Greedy Algorithm — Maintenance Prioritization	
Problem	The MAINTENANCE_LOG table has rows in insertion order. URGENT items can be buried below LOW priority items. A technician
Solution	The greedy approach assigns a local-optimal score to each row: URGENT=40, HIGH=30, NORMAL=20, LOW=10 (priority) + PEN
Result	One click re-orders the entire maintenance table so the most critical, most urgent, still-open jobs appear at the top. Technician imr
Code	maintenancewindow.cpp: greedyPrioritize() (the 'Auto-Prioritize' button)

Every DSA choice above was driven by a real inefficiency or missing capability in the system. No algorithm was added just to fill a checklist.